

Fruit Detection and Counting Using Deep Learning

G. Ramesh Chandra
Dept. of CSE, VNRVJIET,
Hyderabad, India
rameshchandra_g@vnrvjiet.in

K. Sri Rama Murthy
Dept. of CSE, VNRVJIET,
Hyderabad, India
sriramamurthy_k@vnrvjiet.in

Archana S
Dept. of CSE
VNRVJIET
Hyderabad, India
archxana@gmail.com

Snehitha B
Dept. of CSE VNRVJIET
Hyderabad, India
snehithabhooma004@gmail.com

Nishanth G
Dept. of CSE VNRVJIET
Hyderabad, India
goparajunishanth@gmail.com

Navya M
Dept. of CSE VNRVJIET
Hyderabad, India
maidharkarinavya@gmail.com

Sai Charan R
Computer Science & Engineering
VNRVJIET
Hyderabad, India
charanshashi029@gmail.com

Abstract – Manual counting of fruits to estimate yield is a cumbersome task that requires many laborers and may be prone to human error. Wrong counting may lead to financial losses and field mismanagement. To mitigate this problem, there is a need for technological solutions to count the fruits using computer vision and deep learning techniques. The technological solution using computer vision avoids manual counting errors, optimizes time, reduces the number of laborers needed for manual counting, and provides instant yield predictions. Moreover, counting images may lead to the same fruit being counted twice or more across different images. To avoid this, there is a need for a spatiotemporal solution that avoids counting the same fruit more than once. This paper initially focuses on fruit detection using the YOLO framework. Then it implements an object-tracking module to reduce the risk of counting the same fruit more than once. Finally, the object counter module is developed to count the number of fruits. Our results show that our model achieved 90% accuracy, 0.78 mAP, and 0.73 F1 score, demonstrating significant improvements of 33% in mAP and 71% in F1 score compared to using YOLOv11 alone. Different multi-object tracking methods achieve substantial improvements in MAE for frame counting across all videos. These results show that our system has the potential to improve fruit yield assessment methods.

Keywords –Yield Estimation, Manual Counting, Computer Vision, Human Error, Resource Optimization, Multi-Object Tracking, Accuracy Improvement.

I. INTRODUCTION

Yield estimation for fruits is a major function in agriculture that largely determines pricing strategy, resource utilization and overall farm output. There are many challenges that farmers face while performing this analysis. The key challenges that we focused are Farmers have traditionally been estimating yield with the use of manual counting methods which are highly time-consuming, laborious and also error-prone. Farmers rent their trees to third parties and collect a commission as a percentage of the produce brought in or tree value estimates. Correct calculations of yield are also made more difficult by the challenge of manually assessing fruit quality and quantity. Another important challenge they face is to predict the harvest before collecting

the produce from the trees. Based on the yield estimation the farmer may take a proper decision on planning for improving the yield by various precautionary measures.

To mitigate the above challenges and to overcome the disadvantages involved in traditional methods of fruits detection and counting, technological solutions were introduced using computer vision /machine vision/ image processing techniques supported with machine learning or deep learning. This sort of technological solution reduces human error in counting the fruits, improves time management with more precision. This not only improves the efficiency of the task, but also reduces the cost involved in employing the daily labor. The agricultural field managers can plan for the cost estimation based on the demand and supply. The technological solution serves farms of various sizes/volumes, and it offers a scalable solution for better fruit yield prediction or tracking.

The popular deep learning algorithm, i.e. convolutional neural networks (CNNs) offer robustness in detecting the objects under various illumination conditions, under different types of occlusions, complex natural scenes in the background and under varying weather conditions. Various CNN architectures are good at identifying the intrinsic underlying patterns in the image data. There many existing CNN variant models for identifying the various fruits under diverse backgrounds and environmental conditions. Some of these models are robust in terms of rotation invariance, scaling invariance, translation invariance, illumination invariance and occlusion invariance. More practically the performance of the models depends on the fruits data set containing the images taken with different weather conditions, lighting conditions, varying sizes & shapes, imaged with different backgrounds & complexity of them, various colors of fruits for better generalizing to the near real world situations.

The various models are validated through the fruits of various sets of fruits irrespective of the varying environmental factors. To achieve optimal accuracy and dependability, we apply the model to actual videos and images and fine-tuning it accordingly. This iterative optimization allows for compensation of residual inaccuracy and tweaks the model to deliver best-in-class performance.

With these ongoing developments, the developed model matures to deal with real-time data challenges and presents a consistent and accurate fruit yield estimation tool for varied agricultural applications with spatiotemporal inputs for farmers.

The key contributions are two-fold:

- Byte Tracker is integrated to maintain the track history of the detected fruit.
- Region-based counting approach is used to count the tracked objects.

The later sections in this paper cover related works in section II, the details of the proposed system mentioned in section III, the results given in section IV, and the conclusion and future scope in section V.

II. RELATED WORK

Barnea et al. [1], introduced a colour-agnostic, shape-based framework for 3D fruit detection aimed to enhance agricultural robotic systems. The technique exploits both RGB and range data for shape features analysis for fruits might become invisible behind foliage. The method mitigates illumination and appearance variations since it relies on geometry, thereby improving the efficiency of fruit harvesting robots and enhancing precision agriculture.

Borienne et al. [2], investigate the performance of the network on small objects in large images through tiling. It can be noted that there is not much difference between settings. Rather a selection of the training and validation sets is what gives the best accuracy. This paper focuses on how objects detected can become complex in aggregation, due to tiling, and raises the number of errors throughout detection.

In Zhang et al. [3] the Orange Sort algorithm is proposed as a mechanism for tracking oranges while also addressing double counting and occlusion issues in orchard environments. Two sub-algorithms have been combined: OrangeYolo for fruit detection and OrangeSort for tracking with potential errors count decreased practically. The results show that OrangeSort has the minimum counting error value of 0.081 along with a standard deviation of 0.08 and outperforms traditional algorithms like Sort and DeepSort. The study brings into focus specific tracking region counting strategy and the estimation of the motion displacement as it demonstrates its practical application values in agricultural fields.

Neethi et al. [4], published a research for ResNet-50 for yielding estimation of the mango fruit for early yield estimation and better production planning for farming. The paper is organized into four sections: the explanation of the approach and mathematical equations, followed by an analysis of the results along with a comparison, and concluding with a summary of the findings.

Fangfang et al. [5], developed an innovative method towards the detection and counting of apple fruits with deep learning and trunk tracking methods in an orchard. In this

work, the detection network YOLOv4-tiny is combined with a CSR-DCF tracking algorithm for automatic counting of apples during harvest time with accuracy of more than 91%. The approach also detected trunks were designated as the object of tracking and counting to exclude errors due to similarity among the fruits.

Hanwen et al. [6], proposed a fast implementation framework for real-time apple detection in orchards using deep learning. The proposed fruit detector, LedNet, reached notable performance metrics, including a recall of 0.841 with the resnet-101 backbone, and 0.821 recall and 0.853 accuracy with a lightweight backbone.

Manya et al. [7], proposed a paper that investigates tomato fruit detection in greenhouses using the MaskRCNN deep learning algorithm. The experimental results demonstrate that MaskRCNN achieves a mean average precision of 0.82 for detecting tomatoes, surpassing prior metrics of precision (0.88) and recall (0.80) from earlier studies. The architecture ResNext 101 consistently outperforms ResNet 50 and 101, indicating better precision and recall metrics.

J. Lu et al. [8], proposed a new approach for citrus fruit detection inside tree canopies with changing illumination and occlusion conditions. It applies a preliminary segmentation technique combining chromatic aberration mapping and normalized red component mapping to find visible fruit parts. This work thus provides the necessary insight toward robotic harvesting by establishing a partial order relation among overlapping fruit targets that is very significant in robotic path planning.

Longsheng Fu et al. [9], developed a machine vision system for apple detection in dense-foliage fruiting-wall trees using RGB and depth features for robotic harvesting. The authors employed a Kinect V2 sensor for background object removal and Faster ReCNN architectures, resulting in an improved accuracy of 2.5% with very close processing speeds for the datasets of images.

Z. Song et al. [10] introduced the canopy segmentation and wire reconstruction of kiwifruit harvesting through DeepLabV3+ models. The authors used a total of 327 RGB images collected during three harvest seasons, with annotations of four classes: calyx, branch, wire, and background. The work brings out the ability of machine vision to improve efficiency in robotic harvesting by overcoming occlusion and variable fruit visibility in the canopy.

Jing Hu et al. [11] proposed a method to improve fruit yield estimation using a novel model combining Vision Transform with Yolov7. This research addresses the problems in agriculture such as occlusion, overlapping, and light differences that limit the accuracy of existing CNN-based object detection models. This paper illustrates the actual potential of the Yolov7-CA model and advanced monitoring techniques for improving fruit inspection and optimizing transportation and storage strategies in the fruit industry.

A.B. Payne et al. [12] presented an approach for assessing mango crop yield through counting mango fruits of day time images of individual trees. It is done with the study that takes images of fifteen mango trees from all four sides before three weeks of harvest, along with manual fruit counts done to compare. A strong correlation between machine vision counts and manual counts was observed; R^2 was 0.91 for the two sides. Mango counts per image were counted using resultant blobs. The study also discusses potential methods to minimize these limitations, highlighting the feasibility of machine vision for mango yield estimation.

Jun Lu et al. [13] developed a method for accurately detecting and counting immature citrus fruits within tree canopy images which helps farmers in yield estimation, profit analysis and planning regarding application of nutrients. Positive predictions from this classifier were further processed as fruit candidates, with hierarchical contour maps generated around these areas and analyzed using the Circular Hough Transform for shape fitting.

Zhenglin Wang et al. [14] developed a method for in-field mango fruit sizing, which is essential for estimating fruit maturation, size distribution, harvest timing, and marketing needs. The method achieved an RMSE of 4.9 mm and 4.3 mm for length and width estimation, respectively, compared to manual measurements.

H.Gan et al. [15] developed a method for detecting immature green citrus fruit, a critical step for early-stage yield mapping, using a multimodal imaging platform combining color and thermal images. This study utilized thermal imaging under optimized conditions to improve detection accuracy. The integrated system combined image registration, information fusion, and classification into a single step for real-time processing.

Feng Xiao et al. [16] reviewed advancements in fruit detection and automatic harvesting driven by machine learning (ML) and deep learning (DL). The study highlights the strength of DL, particularly convolutional neural networks (CNNs), in extracting high-dimensional features from fruit images, achieving human-level accuracy and speed in specific agricultural tasks.

R. Kestur et al. [17], detected a methodology for the yield estimation of mangoes under pre-harvest conditions using a method for counting and detection from RGB images. The proposed approach uses MangoNet, for mango semantic segmentation and contour-based connected object detection. MangoNet did well compared to variant architectures of FCN trained on similar dataset, showing its applicability in yield estimation for mango.

H. Kuang et al. [18] have provided a new methodology for multi-class fruit detection that uses the effectiveness of the region selection from the image along with improved object proposals. This methodology uses five different complementary features which include LBPs, HOGs, GaborLBP, global color histograms, and global shape features to increase detection accuracy.

Maryam et al. [19], propose yield estimation using simulated deep convolutional neural network, trained on synthetic data and real-world tested data. The strategy incorporates a modified Inception-ResNet architecture to capture features across different scales, to allow the efficient counting of fruits even with shadows, occlusions by foliage or branches, and overlap among fruits.

H. Mirhaji et al. [20] proposed an estimation approach of fruit yield in precision agriculture based on YOLO detection models. The suggested YOLO model is employed for mature Dezful native orange detection and counting using images taken with different illumination conditions. Two-side and four side imaging show notable differences in yield estimation, which results in planning to mix two-side imaging for thin canopies and four-side imaging for dense canopies.

Existing methods for fruit counting and detection are plagued by occlusion, overlapping, varying illumination, and inconsistent tracking, leading to missed or redundant counts. The majority of the methods employ static images or appearance features, which limit their performance in dynamic scenarios. This method integrates YOLO for real-time detection with ByteTracker for robust multi-frame tracking and hence reduces ID switching and double counting. Region-based counting and motion analysis also enhance accuracy when there is dense vegetation, and the system can be fielded in Agri-applications.

III. METHODOLOGY

3.1. General Methodology of Working Model

The proposed system is intended to identify and count fruits from two types of input feeds. It employs the use of a combination of deep learning technologies, that is, YOLO11 for object detection and ByteTracker for object tracking. The system can process video feeds in real-time, detect individual fruits, track their movement between frames, and count them. The overview of the system is depicted in Fig. 1.

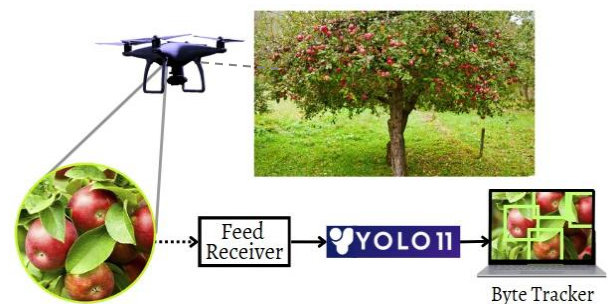


Fig. 1: Overview of the System

(i) Feed Receiver

The feed receiver handles the capturing of the video input for the system. The source of the video feed can include real-time drones, live streaming cameras, and uploaded stored ones. In case of real-time feeds, they may be sourced from a drone that captures top views of trees in an orchard or a farm. Alternatively, the system can be used with stored video footage of fruits taken from any camera. This gives the system flexibility to be used in any environment, whether it

is real-time monitoring of fruits in a field or processing pre-recorded videos to count fruits for quality assurance in a warehouse. After obtaining the video feed, it is processed frame by frame for the subsequent steps.

(ii) YOLO11

YOLO is an acronym for ‘You Only Look Once’. YOLO11 is a fast and effective algorithm for detecting objects. It is implemented on the frames generated from the real-time feed or a pre-recorded video. As each frame is processed, the fruits are detected and bounding boxes are generated around them. It then labels the class of each bounding box and provides a confidence score for each object detected. Even in case of fruit occlusions, YOLO11 can accurately identify and locate the fruits, making it an integral part of the proposed system.

(iii) ByteTracker

After YOLO11 detects the fruits in each frame, ByteTracker takes over to track these fruits as they move across frames. ByteTracker uses data from YOLO to keep track of the same fruits across different frames. It assigns a unique ID to each fruit, ensuring that the same fruit is not counted multiple times or confused with another. Even when fruits move or become occluded (hidden behind other objects), ByteTracker ensures that the system continues to track them accurately. It also integrates a counting algorithm to tally the tracking. This is critical for long-term fruit tracking in a video feed, as the system can maintain a consistent count of the fruits, regardless of how they move or overlap with each other.

3.2 Detailed Methodology of the Proposed System

A detailed methodology of the proposed system is depicted in Fig. 2.

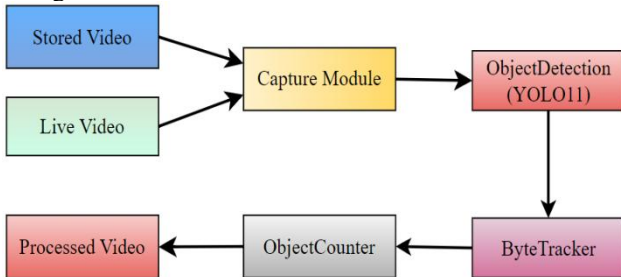


Fig. 2: Detailed methodology of the Proposed System

A. Capture Module

The Capture Module is responsible for acquiring video footage, which serves as input for the apple detection and tracking system. The input is obtained from two primary sources:

(i) Real Time Drone Video Feed

A 360-degree rotating drone forms a circle around an apple tree that records live video from various angles. Due to its high-resolution camera, the drone is capable of easily observing apples under different lighting conditions. Real-time video streaming to the processing module allows for immediate tracking and detection.

(ii) Stored Video Footage

The system also supports pre-recorded videos read from a stored file using OpenCV and the video properties such as

frame width, height, and frame rate are extracted for processing. These parameters ensure that the system adjusts to different video formats. Before their deployment on live feed drone streams, these videos help optimize the YOLO11 model and tracking algorithms. Frame-by-frame processing is done on the video that has been stored.

The captured frames from both sources are passed to the detection module.

B. Object Detection

The Object Detection module identifies apples in video frames captured from a drone feed. It does this by using YOLO11, the most advanced deep learning model, to detect and locate objects in real time. It extracts all important visual features like color, shape, and texture to distinguish apples from leaves, branches, or other objects. The model can identify various fruits based on these properties. This purely model based algorithm will detect any type of fruit once trained with its data. We can train the model with the type of fruit we are trying to detect. Since it has been trained over a custom dataset on apples, this model is specialized towards detecting apples under conditions such as different angles, occlusions, lighting variations.

(i) Dataset of Training Apple Images

A diverse dataset of fruit images and videos is curated from various agricultural settings which contains thousands of images of apples that were used to train the model. The dataset was annotated using bounding boxes to mark apples in images. The model was trained using transfer learning on a pre-trained YOLO11 base model.

(ii) Object Detection

After receiving a video frame from the Capture Module, the YOLOv11 model processes the frame and identifies apples. A region of interest (ROI) is defined to restrict detection to relevant areas in the video which ensures that detection occurs within the full frame, but it can be adjusted to focus on specific areas if needed. It generates bounding boxes around detected apples. Each detected apple is assigned a confidence score. Apples are rejected with less than a given confidence value (for example, 70%). After detection, apples are assigned unique tracking IDs, which are then used in the Tracking Module for further processing. Algorithm 1 describes the object detection using YOLO.

Algorithm 1: Algorithm for Object Detection using YOLO

Input: input_frame

Initialization:

Load YOLO model with pre-trained weights and config files
Define class labels (e.g., apple.)
Set detection threshold and non-max suppression (NMS) threshold

Detection:

Pass input_frame to the YOLO model
Obtain raw predictions: bounding boxes, confidence scores, and class probabilities
Filtering:
For each prediction:

If confidence score > threshold:

Extract bounding box coordinates: (x, y, width, height)
Get class label with the highest score

Apply Non-Max Suppression (NMS) to remove overlapping boxes

Output: List of detected objects with format:

[class_label, confidence_score, bounding_box_coordinates]

C. Tracking Processor

The key application of the object tracker is the counting of apples over frames. This tracker can be implemented using various approaches but there is one tracker that suits our use case better than any. Various trackers are employed to check their compatibility with the model and their accuracy in performing the job. From them ByteTracker is considered as the most compatible option that best fits the job scheduled. ByteTracker has higher accuracy in tracking objects with similar properties and has a plugin that is developed inside YOLO to connect and operate with the model directly. Various trackers that we used in our search for best trackers are mentioned below.

i. SORT (Simple Online Real-Time Tracker)

SORT is an efficient and lightweight tracking algorithm that uses Kalman Filters for motion prediction and the Hungarian Algorithm for data association. This algorithm performs well in non-occluded environments but fails with occluded environments, leading to frequent ID switches. A more robust tracker is required for real-world orchard conditions because of these limitations.

ii. DeepSORT (Deep Simple Online Real-Time Tracker)

DeepSORT extends SORT by incorporating deep learning-based appearance descriptors for better re-identification of objects across frames. It enhances the performance in the case where the object is temporarily out of view (occlusion). In this context DeepSORT is not very efficient in the case of smaller objects, like apples because it requires a lot of visual information to create feature embeddings. The computational cost is more which rules out its real-time usability in the systems with limited resources.

(iii) ByteTracker (Byte-Track Improved SORT)

ByteTracker refines SORT by removing low-confidence detections because the fruit images can be detected as small objects, which normally have a lower detection score. Tracking apples in a complex environment has its own difficulties such as: Occlusion, as apples can be covered by leaves or branches. Small object size is difficult for the conventional trackers to maintain consistent IDs. ByteTrack overcomes these issues by employing a two-stage association strategy:

- Primary Association:* Associate high-confidence detection to tracks using the motion estimation (Kalman Filters)
- Secondary Association:* Incorporate the low confidence detections without losing apples due to occlusion or weak detections.

This reduces the ID switches significantly and is much more robust for dense tracking in orchards. Furthermore, it still is computationally efficient enough that it remains the choice for this.

Algorithm 2 describes tracking of objects.

Algorithm 2: Algorithm for Tracking objects using ByteTracker

Input : detection_results (from YOLO)

Initialization:

Create Empty track_list = []

Declare Kalman filter parameters:

State vector $x = [cx, cy, a, h, vx, vy, va, vh]^T$

(center_x, center_y, aspect_ratio, height, velocities)

Define high_thresh, low_thresh for ByteTrack filtering

Define IOU threshold for association

COMPUTE:

high_conf_detections (confidence $\geq$ high_thresh)

low_conf_detections (low_thresh $\leq$ confidence $<$ high_thresh)

Predict Track States (Kalman Filter Prediction)

$$x^k = F \cdot x^{k-1}$$

$$P^k = F \cdot P^{k-1} \cdot F^T + Q$$

Where

F is motion function

Q is additional bias

P is uncertainty in previous prediction

Data Association (Hungarian Algorithm using IOU distance)

Compute IOU matrix between predicted track boxes and high_conf_detections

Solve:

$$\min_{i,j} \sum c_{ij} \cdot x_{ij}$$

Where:

- $c_{ij} = 1 - \text{IOU}(\text{track}_i, \text{det}_j)$
- $x_{ij} \in \{0, 1\}$, $x_{ij} = 1$ if detection j is assigned to track i

Update Tracks (Kalman Filter Update)

$$K = P^k H^T (H P^k H^T + R)^{-1}$$

$$x^k = x^{k-1} + K(z^k - Hx^{k-1})$$

$$P^k = (I - KH)P^{k-1}$$

OUTPUT: tracked_objects in the format of [track_id, class_label, confidence_score, bounding_box_coordinates]

D. Object Counter

The Object Counter module is responsible for estimating the total number of apples on the tree by integrating the tracking data from ByteTracker with a counting algorithm. This module ensures that each apple is counted only once, preventing overcounting due to movement, occlusion, or reappearance in different frames.

The difficulties in counting apples from a rotating drone video include Occlusions, Duplicate Counting, Motion Variability, Lighting and Shadows. The Object Counter overcomes all these with the use of Tracking based counting.

(i) Integration of the Counting Algorithm

The algorithm for counting analyzes the tracking history of each apple that has been detected in consecutive frames. Whenever an apple first appears, the tracking algorithm assigns it a unique ID (ByteTracker). The tracking processor retrieves object details, such as bounding boxes, tracking IDs, and class labels; the counting algorithm checks whether it is a new tracking ID before actually counting each detected apple, meaning that the same apple, which has been detected several times, will not be counted multiple times. If an apple stays in the frame, its ID is maintained so that it is not counted twice. If an apple goes out of view for a few frames and then reappears, the tracker tries to reconnect it with the same ID rather than assigning a new one. The count is only updated when the apple is found to be unique based on track ID and location.

(ii) Output of the Object Counter

- Real-time Apple Count display
- Processed Video with Tracking IDs
- For every frame processed, the count of apples is updated on-screen.

Algorithm 3 discuss the procedure for counting objects.

Algorithm 3: *Algorithm for Counting objects*

Input: current_centroid, track_id, prev_position, cls

Initialization:

Skip processing if prev_position is None or track_id is already in counted_ids.

Let the polygon be defined by a set of vertices:

$\{P(x_1, y_1), P_2(x_2, y_2), \dots, P_n(x_n, y_n)\}$, $n > 2$

Let the object's position be:

$p = \text{prev_position}$, $c = \text{current_centroid}$

Check if the object has entered the polygon region

$\text{polygon.contains}(c) = \text{True}$

Compute:

$\text{region_width} = \max(x_i) - \min(x_i)$

$\text{region_height} = \max(y_i) - \min(y_i)$

Vertical Region if $\text{region_width} < \text{region_height}$:

Use x-direction:

if $p[0] > c[0]$: IN, else OUT

Horizontal Region if $\text{region_width} \geq \text{region_height}$:

Use y-direction

if $p[1] > c[1]$: IN, else OUT

Update per-class counts:

Add track_id to counted_ids

Output: Updated in_count, out_count, counted_ids

IV. RESULTS

4.1 Dataset and Model Training

The model is trained using the dataset: *Fruits-360 dataset* (<https://www.kaggle.com/datasets/moltean/fruits>): A dataset of images containing fruits, vegetables, nuts and seeds. The dataset has 126305 images of different fruits and more. Out of which, 7571 images are filtered out for training and testing the model. Each image is annotated in YOLOv11 format. Each image was augmented with random rotation, cropping (up to 55%), and Gaussian blur to improve model performance. The dataset is divided into training (6,056 images, 80%), validation (605 images, 8%), and test (910 images, 12%) sets for effective learning and evaluation.

The model was trained for 200 epochs using input images resized to 640×640 pixels and halted at 150 epochs to prevent overfitting. A batch size of 4 was employed, with the workers parameter set to 4 to enable parallel data loading. The training utilized the default learning rate of 0.01. The training was done on an NVIDIA RTX 4060 GPU.

Fig. 3 shows the Training loss curves and Fig. 4 shows the validation loss curves. The training loss and validation loss initially decrease, indicating good learning. But the validation loss plateaus at epoch 150 while the training loss continues to decrease, indicating overfitting. To prevent this, the training is terminated at epoch 150 because additional training can degrade generalization performance on new data.

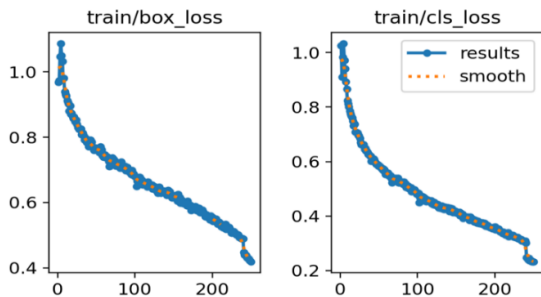


Fig. 3. Training loss curves for classification and prediction

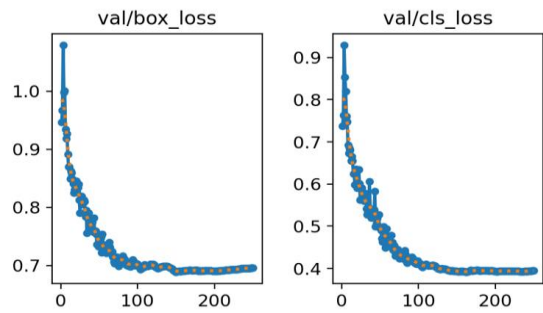


Fig. 4. Validation loss curve for classification and prediction

4.2 Comparison of models

The Table 1 compares computational complexity and efficiency with the proposed model and the traditional YOLO variants. The YOLO11-fv model has the same number of parameters and inference rate as that of YOLO11m because it is architecturally derived from the YOLO11m configuration. The proposed model is faster in processing than smaller models and less complex than larger models in a balanced trade-off between speed and performance.

Table 1: Complexity and efficiency of YOLO variants

YOLO model	Speed CPU ONNX (ms)	Params (M)	Flops (B)
YOLO11n	56.1 $\pm$ 0.8	2.6	6.5
YOLO11s	90.0 $\pm$ 1.2	9.4	21.5
YOLO11m	183.2 $\pm$ 2.0	20.1	68.0
YOLO11l	238.6 $\pm$ 1.4	25.3	86.9
YOLO11x	462.8 $\pm$ 6.7	56.9	194.9
YOLO11-fv	210.4 $\pm$ 0.5	20.05	34.09

The Table 2 demonstrates that the custom-trained YOLO model (yolo11-fv) outperforms all YOLO11 variants in every metric. The model shows strong performance in detection of apples, achieving 0.959 precision, 0.588 recall, and an F1 score of 0.729, indicating effective and balanced detection. It also records a mAP@0.5 of 0.779 and mAP@0.5:0.95 of 0.700, confirming its robustness in localizing apples under various conditions.

Table 2: Performance metrics of yolo variants

YOLO model	Precision	mAP50	F1 Score
YOLO11n	0.932	0.551	0.278
YOLO11s	0.897	0.575	0.406
YOLO11m	0.901	0.572	0.378
YOLO11l	0.873	0.586	0.428
YOLO11x	0.854	0.568	0.422
YOLO11-fv	0.959	0.779	0.729

4.3 Tracking and Counting Efficiency

This section provides a brief explanation on how the tracking and counting efficiency is measured. It emphasizes the consistency of object tracking across consecutive frames and

quantifies the tracking loss. It also explains how the tracking directly influences the counting of fruits.

To evaluate the effectiveness of our tracking system, tracking metrics were recorder at every 30-frame interval over a test sequence of 300 frames were tracked. For each frame , key parameters were monitored and recorded including valid and retained IDs , efficiency ,lost IDs and ID switches. The results are achieved by using the model which outperformed YOLO. The system might behave differently when used with different model. Table 3 represents the results of tracking efficiency across the frames.

Table 3: Tracking Efficiency across frames

Frame	Valid IDs	Retained IDs	Efficiency (%)
30	15	14	93.33333
60	18	16	88.88889
90	20	19	95
120	17	15	88.23529
150	19	18	94.73684
180	21	20	95.2381
210	23	21	91.30435
240	22	21	95.45455
270	24	23	95.83333
300	25	24	96

The system demonstrated consistent performance, with the number of valid apple detections ranging between 15 and 25 per frame. On average, tracking efficiency remained above 90% with an average efficiency of 93.4% , indicating that the majority of apples were retained across frames with consistent IDs validating the robustness of tracking pipeline.

The IDs are considered lost when they are not detected again at the end of the region or were no longer detected in subsequent frames. The IDs are considered switched when a tracked apple was reassigned a different ID partway through its trajectory. The count of lost fruits remained low across all samples, with most frames reporting between 1-2 lost apples. Notably, the lost ratio was highest at just 12.5%, showcasing the robustness of the tracking algorithm in maintaining identity continuity. Additionally, the number of ID switches-where an apple was assigned a different ID in subsequent frames-remained minimal, typically 0 or 1, confirming identity stability. Table 4 show the Lost IDs and ID switches across frames.

Table 4: Lost IDs and ID switches across frames

Frame	Valid IDs	Lost	ID Switches
30	15	1	1
60	18	2	2
90	20	1	1
120	17	2	0
150	19	1	1
180	21	1	2
210	23	2	1
240	22	1	1

270	24	1	0
300	25	1	1

In the system, fruit counting is performed by tracking each detected fruit across frames using unique identifiers (IDs). The system counts a fruit only when its ID is retained and crosses a defined region of interest (ROI) with some special conditions mentioned in methodology section. Therefore, the accuracy of the counting mechanism is highly dependent on the consistency and reliability of the tracking algorithm.

Any loss or switch in ID will result in either an undercount i.e. if a fruit is no longer tracked before reaching the ROI or an overcount (if the same fruit is assigned multiple IDs). The counting efficiency is therefore directly proportional to that of tracking. With high tracking efficiency-reported in retained IDs and low switch/loss rates-it can be guaranteed that the majority of fruits are being counted correctly, once, and at the right time. The accuracy of counting in the system therefore closely follows the tracking performance, both with average efficiencies of over 90%.

4.4 Evaluation metrics

This section provides a comprehensive evaluation of the model's performance, highlighting its strengths in precision, recall, and detection accuracy while also addressing areas for refinement.

The Precision-Recall curve shows the performance of the object detection model in identifying apples and other potential classes. The Average Precision for the "Apple" class and the mean Average Precision across all classes at an IoU of 0.5 are both 0.780, indicating a good balance between precision and recall. Fig. 5 shows precision-recall curve

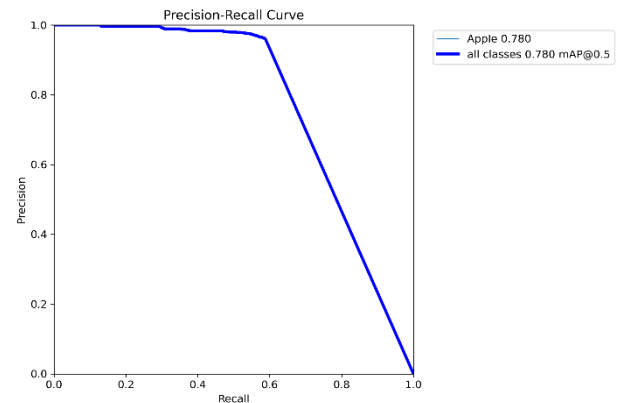


Fig. 5: Precision-Recall Curve

4.5 Execution Results

The results presented in this section were obtained using a trained yolo model that outperformed existing YOLO models in terms of stability and identity preservation. It is important to note that the system's behavior and performance may vary when integrated with different detection models, and tracking robustness could depend on the quality and consistency of detections provided. Fig. 6 shows the detection of fruits across frames. Subsequently Fig. 7 shows the counting of the fruits across the frames.

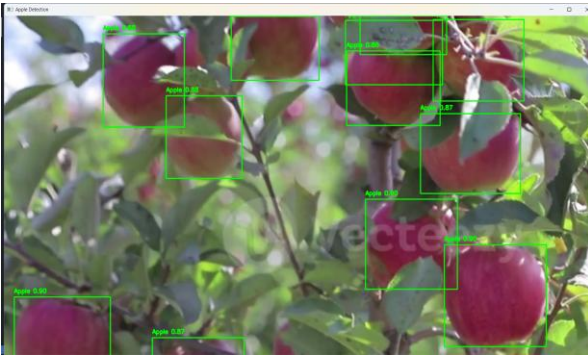


Fig 6: Detection of fruits across frames



Fig 7: Counting the fruits across frames

V. CONCLUSION

This work presents a cost-effective approach to automate fruit detection and counting with YOLOv11 for object detection and ByteTracker for tracking. The proposed system is able to reduce the error in counting the fruits more than once with meticulous pipeline workflow, where it initially detects fruits, the track them with a subsequent counting module. This pipeline is successful in identifying fruits from dense tree setting obtained through real time drone videos. The accurate yield estimation of fruits improves farming decisions. The proposed system can be deployed for identifying and counting the different varieties of fruits, but the success of such models depends on how well the respective data set is.

Even though the proposed model performs well for varying conditions, it may not be very effective in extreme weather conditions with heavy rain, dense fog and high winds. Still the more robust algorithms need to be developed to handle densely overlapped fruits in the natural scenes, otherwise the fruit counting may lead to the yield. To mitigate the said problem the latest techniques like vision transformers, attention-based mechanisms etc. may be employed. Further, the robust models can be extended to multiple fruit and hybrid farming too.

REFERENCES

- [1] Barnea, Ehud, Rotem Mairon, and Ohad Ben-Shahar. "Color-agnostic shape-based 3D fruit detection for crop harvesting robots." *Biosystems Engineering* 146 (2016): 57-70.
- [2] Borianne, Philippe, Frédéric Borne, Julien Sarron, and Émile Faye. "Deep Mangoes: from fruit detection to cultivar identification in colour images of mango trees." *arXiv preprint arXiv:1909.10939* (2019).
- [3] Zhang, Wenli, Jiaqi Wang, Yuxin Liu, Kaizhen Chen, Huibin Li, Yulin Duan, Wenbin Wu, Yun Shi, and Wei Guo. "Deep-learning-based in-field citrus fruit detection and tracking." *Horticulture Research* 9 (2022): uhac003.
- [4] Vasanth, N. Managali, and R. PPandian. "Fast region based convolutional neural network ResNet-50 model for on tree Mango fruit yield estimation." *Indonesian Journal of Electrical Engineering and Computer Science* 33, no. 2 (2024): 1084-1091.
- [5] Gao, Fangfang, Wentai Fang, Xiaoming Sun, Zhenchao Wu, Guanao Zhao, Guo Li, Rui Li, Longsheng Fu, and Qin Zhang. "A novel apple fruit detection and counting methodology based on deep learning and trunk tracking in modern orchard." *Computers and Electronics in Agriculture* 197 (2022): 107000.
- [6] Kang, Hanwen, and Chao Chen. "Fast implementation of real-time fruit detection in apple orchards using deep learning." *Computers and Electronics in Agriculture* 168 (2020): 105108.
- [7] Afonso, Many, Hubert Fonteijn, Felipe Schadeck Fiorentin, Dick Lensink, Marcel Mooij, Nanne Faber, Gerrit Polder, and Ron Wehrens. "Tomato fruit detection and counting in greenhouses using deep learning." *Frontiers in plant science* 11 (2020): 571299.
- [8] Lu, Jun, and Nong Sang. "Detecting citrus fruits and occlusion recovery under natural illumination conditions." *Computers and Electronics in Agriculture* 110 (2015): 121-130.
- [9] Fu, Longsheng, Yaqoob Majeed, Xin Zhang, Manoj Karkee, and Qin Zhang. "Faster R-CNN-based apple detection in dense-foilage fruiting-wall trees using RGB and depth features for robotic harvesting." *Biosystems Engineering* 197 (2020): 245-256.
- [10] Song, Zhenzhen, Zhongxian Zhou, Wenqi Wang, Fangfang Gao, Longsheng Fu, Rui Li, and Yongjie Cui. "Canopy segmentation and wire reconstruction for kiwifruit robotic harvesting." *Computers and Electronics in Agriculture* 181 (2021): 105933.
- [11] Hu, Jing, Chuang Fan, Zhoupu Wang, Jinglin Ruan, and Suyin Wu. "Fruit detection and counting in apple orchards based on improved YOLOv7 and multi-Object tracking methods." *Sensors* 23, no. 13 (2023): 5903.
- [12] Payne, Alison B., Kerry B. Walsh, P. P. Subedi, and Dennis Jarvis. "Estimation of mango crop yield using image analysis—segmentation method." *Computers and electronics in agriculture* 91 (2013): 57-64.
- [13] Lu, Jun, Won Suk Lee, Hao Gan, and Xiuwen Hu. "Immature citrus fruit detection based on local binary pattern feature and hierarchical contour analysis." *Biosystems Engineering* 171 (2018): 78-90.
- [14] Wang, Zhenglin, Kerry B. Walsh, and Brijesh Verma. "On-tree mango fruit size estimation using RGB-D images." *Sensors* 17, no. 12 (2017): 2738..
- [15] Gan, Hao, Won Suk Lee, Victor Alchanatis, Reza Ehsani, and John K. Schueller. "Immature green citrus fruit detection using color and thermal images." *Computers and Electronics in Agriculture* 152 (2018): 117-125.
- [16] Xiao, Feng, Haibin Wang, Yueqin Xu, and Ruiqing Zhang. "Fruit detection and recognition based on deep learning for automatic harvesting: An overview and review." *Agronomy* 13, no. 6 (2023): 1625.
- [17] Kestur, Ramesh, Avadesh Meduri, and Omkar Narasipura. "MangoNet: A deep semantic segmentation architecture for a method to detect and count mangoes in an open orchard." *Engineering Applications of Artificial Intelligence* 77 (2019): 59-69.
- [18] Rahnmooonfar, Maryam, and Clay Sheppard. "Deep count: fruit counting based on deep simulated learning." *Sensors* 17, no. 4 (2017): 905.
- [19] Kuang, Hulin, Cairong Liu, Leanne Lai Hang Chan, and Hong Yan. "Multi-class fruit detection based on image region selection and improved object proposals." *Neurocomputing* 283 (2018): 241-255.
- [20] Mirhaji, Hamzeh, Mohsen Soleymani, Abbas Asakereh, and Saman Abdanan Mehdizadeh. "Fruit detection and load estimation of an orange orchard using the YOLO models through simple approaches in different imaging and illumination conditions." *Computers and Electronics in Agriculture* 191 (2021): 106533.